

Embeder MCP Server Architecture

The Model Context Protocol (MCP) forms the foundational communication backbone of **Embeder**, enabling language models to safely interact with local hardware toolchains, compilers, simulators, and CAD automation tools.

1. Core MCP Servers Breakdown

MCP Server Name	Exposed Tools / Resources	Description
Datasheet & Component Server	<code>`search_pinouts()`</code> , <code>`get_register_map()`</code> , <code>`lookup_component_bom()`</code>	Queries local vector indices of MCU datasheets, register specs, and peripheral configurations (Arduino, ESP32, STM32, RP2040).
Firmware Compilation Server	<code>`compile_firmware()`</code> , <code>`get_compiler_errors()`</code> , <code>`flash_target()`</code>	Executes background build toolchains (<code>`avr-gcc`</code> , <code>`esp-idf`</code> , <code>`arm-none-eabi-gcc`</code>) and formats build errors for model context reflection.
Simulation & Validation Server	<code>`run_renode_simulation()`</code> , <code>`capture_waveform()`</code> , <code>`check_timing()`</code>	Launches instruction-level CPU simulations (Renode/QEMU) and returns register snapshots and UART/GPIO output logs.
PCB Layout & EDA Server	<code>`generate_netlist()`</code> , <code>`route_traces_kicad()`</code> , <code>`check_drc()`</code> , <code>`export_gerber()`</code>	Invokes Python automation scripts against KiCad's <code>`pcbnew`</code> module to convert high-level schematics into physical board files.

2. MCP Message Flow Protocol

- Intent Processing:** The user asks Embeder to design an ESP32 Bluetooth temperature logger.
- Context Query:** The AI model calls ``search_pinouts("ESP32-WROOM-32")`` via the Datasheet MCP Server to identify available I2C pins.
- Code Synthesis & Build:** The model generates C++ firmware and invokes ``compile_firmware(target="esp32")`` via the Firmware Compilation Server.
- Verification:** The model runs ``run_renode_simulation()`` to verify memory buffers and peripheral startup sequences.
- EDA Generation:** The model invokes ``generate_netlist()`` and ``route_traces_kicad()`` on the PCB EDA Server to produce a production-ready KiCad PCB layout.